

OCaml - Predavanje 6

Tail Recursion, Exceptions, First-Class Functions

1. Tail Recursion (Repna rekurzija)

Obična rekurzija gradi lanac poziva na stack-u, svaki poziv čeka da se prethodni završi. Kod **tail recursion**, rekurzivni poziv je **zadnja operacija** u funkciji, pa kompajler može ponovo koristiti isti stack frame umjesto da alocira novi.

Zasto je bitno: Obična rekurzija može izazvati **stack overflow** za velike ulaze (npr. $n = 100000$), dok tail-rekurzivna verzija radi bez problema.

Tehnika: Koristimo **akumulator** (`acc`) pomoćnu varijablu koja nosi medijerezultat kroz rekurziu.

2. Exceptions (Izuzeci)

Izuzeci omogućavaju rukovanje greskom bez da se funkcija prekine nekontrolisano.

Definicija izuzetka:

```
exception MyException of int * string;;
```

- `exception` - ključna riječ za definisanje novog izuzetka
- `of int * string` - izuzetak može nositi podatke (broj i poruku)

Bacanje izuzetka:

```
raise(MyException(n, "n je negativan!"))
```

Hvatanje izuzetka (try...with):

```
try napravi_list_tail_rec(-5) with MyException(broj, poruka) -> [];;
```

- `try izraz with` - pokušava izvršiti izraz
- `MyException(broj, poruka) -> []` - ako se baci `MyException`, hvata ga i vraća praznu listu
- Pattern matching radi unutar `with` - možemo izvuci podatke iz izuzetka

3. First-Class Functions (Funkcije kao vrijednosti)

U OCaml-u, funkcije su **first-class citizens** - mogu se:

- Cuvati u varijablama
- Proslijediti kao argumenti drugim funkcijama
- Vratiti kao rezultat iz funkcije
- Cuvati u listama i drugim strukturama podataka

Anonimne funkcije (lambda):

```
fun x -> x * 2          (* funkcija koja udvostučuje broj *)
fun x -> (x mod 2) = 0   (* funkcija koja provjerava parnost *)
```

Funkcije koje vraćaju funkcije:

```
let translacija_pom((a:float), (b:float)) =
  let translacija_sa_odredjenim_parametrima({x=tx; y=ty}: tacka) = translacija(t, a, b)
  in translacija_sa_odredjenim_parametrima;;
```

Ovo je tehnika **partial application** - fiksiramo neke parametre i dobijemo novu funkciju koja prima preostale.

Lista funkcija:

```
(tacka -> tacka) list    (* lista transformacijskih funkcija *)
```

Funkcije se mogu cuvati u listama i primjenjivati redom.

Zadaci

Zadatak 1 — Generisanje liste (tail recursion i exceptions)

Implementirati funkciju koja prima prirodan broj `n` i vraća listu brojeva od 1 do `n`. Implementirati jednu verziju koja koristi tail recursion i drugu koja ne koristi. Uporediti ponašanje za jako veliko `n` (npr. `n = 100000`). Ako je proslijedjeni broj manji od 1, baciti izuzetak. Poziv funkcije smjestiti u `try` izraz.

Rjesenje:

Losa verzija (bez tail recursion):

```
let rec napravi_list_losa(n: int) = if n = 1 then [1] else napravi_list_losa(n-1)@[n];;
```

Problem: Svaki rekurzivni poziv mora sačekati rezultat, pa se dodaje `@[n]` tek nakon što se vrati. Za `n = 100000` može izazvati stack overflow. Dodatno, operator `@` prolazi cijelu listu svaki put — kompleksnost $O(n^2)$.

Dobra verzija (tail recursion + exception):

```
exception MyException of int * string;;

let rec napravi_list_tail_rec(n: int) =
  if n < 0 then raise(MyException(n, "n je negativan!")) else
    let rec pomocna((n:int), (acc:int list)) = if n = 0 then acc else pomocna(n-1, n::acc)
    in pomocna(n, []);;
```

Idea: Pomocna funkcija `pomocna` gradi listu od `n` prema 1, dodajući `n::acc` (na pocetak — $O(1)$ operacija). Kad `n` dodje do 0, vraća akumulator. Rekurzivni poziv `pomocna(n-1, n::acc)` je **zadnja operacija** — tail recursion.

Poziv sa hvatanjem izuzetka:

```
try napravi_list_tail_rec(-5) with MyException(broj, poruka) -> [];;
```

Poredjenje:

<code>napravi_list_losa(10000);;</code>	<code>(* radi, ali sporo *)</code>
<code>napravi_list_tail_rec(10000);;</code>	<code>(* radi brzo *)</code>
<code>napravi_list_losa(100000);;</code>	<code>(* stack overflow! *)</code>
<code>napravi_list_tail_rec(100000);;</code>	<code>(* radi bez problema *)</code>

Zadatak 2 — Obrtanje liste (tail recursion)

Implementirati funkciju koja prima listu i vraća listu u obrnutom redoslijedu. Implementirati jednu verziju koja koristi tail recursion i drugu koja ne koristi.

Rjesenje:

Losa verzija:

```
let rec obrni_losa(l: int list) =
  match l with
  | [] -> []
  | x::tail -> obrni_losa(tail) @ [x];;
```

Problem: `obrne_losa(tail) @ [x]` — rekurzivni poziv nije zadnja operacija (nakon njega ide `@ [x]`). Kompleksnost $O(n^2)$ jer `@` prolazi cijelu listu.

Dobra verzija (tail recursion):

```
let rec obrni_tail_recursion(l: int list) =
  let rec pomocna((l: int list), (acc: int list)) =
    match l with
    | [] -> acc
    | x::tail -> pomocna(tail, x::acc)
  in
  pomocna(l, []);;
```

Idea: Pomocna funkcija skida elemente sa pocetka `l` i dodaje ih na pocetak `acc`. Posto `::` dodaje na pocetak, redoslijed se automatski obrće. Kompleksnost $O(n)$.

Poredjenje:

```
obrni_losa(napravi_list_tail_rec(10000));;          (* radi, ali sporo *)
obrni_tail_recursion(napravi_list_tail_rec(10000));; (* radi brzo *)
```

Zadatak 3 — Transformacije tacaka (first-class functions)

Implementirati tip za tacku, transformacijske funkcije (translacija, skaliranje), te funkcije koje rade sa listama transformacija koristeći first-class functions, map i filter.

Rjesenje:

Tip tacke:

```
type tacka = {x: float; y: float};;
```

Transformacijske funkcije:

```
let translacija(({x=tx; y=ty}: tacka), (a: float), (b: float)) = {x=tx+.a; y=ty+.b};;
```

```
let skaliranje(({x=tx; y=ty}: tacka), (a: float), (b: float)) = {x=tx*.a; y=ty*.b};;
```

Primjena liste transformacija na tacku:

```
let rec uradi_transformacije(({x=tx; y=ty}: tacka), (lista_transformacija: (tacka -> tacka) list)) =
  match lista_transformacija with
  [] -> {x=tx; y=ty}
  | transf::tail -> uradi_transformacije(transf({x=tx; y=ty}), tail);;
```

Ideja: Lista lista_transformacija sadrži funkcije tipa tacka -> tacka. Primjenjujemo prvu transformaciju na tacku, pa rekurzivno primjenjujemo ostatak na rezultat.

Nadovezivanje transformacija (vraca funkciju):

```
let rec nadovezi_transformacije(lista_transformacija: (tacka -> tacka) list) =
  let f (t: tacka) = uradi_transformacije(t, lista_transformacija) in
  f;;
```

Implementacija map i filter:

```
let map((l: 'a list), (f: 'a -> 'b)) =
  let rec pomocna((ostatak: 'a list), (acc: 'b list)) =
    match ostatak with
    [] -> acc
    | x::tail -> pomocna(tail, f(x)::acc)
  in List.rev(pomocna(l, []));;

let filter((l: 'a list), (f: 'a -> bool)) =
  let rec pomocna((ostatak: 'a list), (acc: 'a list)) =
    match ostatak with
    [] -> acc
    | x::tail -> if f(x) then pomocna(tail, x::acc) else pomocna(tail, acc)
  in List.rev(pomocna(l, []));;
```

- map — primjenjuje funkciju f na svaki element liste i vraca novu listu rezultata
- filter — vraca samo elemente za koje funkcija f vraca true
- Obje koriste tail recursion sa akumulatorom i List.rev na kraju (jer :: dodaje na pocetak)

Primjer koristenja:

```
filter([1;2;3;4], fun x -> ((x mod 2) = 0));; (* = [2; 4] *)
```

Zadaci za vjezbu

V1. Napisati tail-rekurzivnu funkciju suma_liste koja prima listu cijelih brojeva i vraca sumu svih elemenata.

V2. Napisati tail-rekurzivnu funkciju `fibonacci(n)` koja vraća n-ti Fibonaccijev broj.

V3. Definirati izuzetak `PraznaLista` i napisati funkciju `sigurni_head` koja vraća prvi element liste, a baca izuzetak ako je lista prazna.

V4. Koristeći `map`, napisati funkciju koja prima listu cijelih brojeva i vraća listu njihovih kvadrata.

V5. Koristeći `filter`, napisati funkciju koja prima listu tačaka i vraća samo one tačke koje su unutar kruga radijusa `r` sa centrom u koordinatnom početku.

V6. Implementirati funkciju `rotacija(t, ugao)` koja prima tačku `t` i ugao u radijanima, te vraća novu tačku rotiranu oko koordinatnog početka. Koristiti formule:
 $x' = x * \cos(\text{ugao}) - y * \sin(\text{ugao})$, $y' = x * \sin(\text{ugao}) + y * \cos(\text{ugao})$.

V7. Implementirati funkciju `rotacija_oko_tacke(t, centar, ugao)` koja prima tačku `t`, centar rotacije `centar` i ugao u radijanima, te vraća novu tačku rotiranu oko zadanog centra. *Hint:* Iskoristiti translaciju i funkciju `rotacija` iz V6 — prvo translirati tačku tako da centar dođe u koordinatni početak, zatim primijeniti rotaciju, pa translirati nazad.